KNet

Release 1.1

Yiding Hao

Aug 23, 2025

CONTENTS

1 API Reference **1**
1.1 dataset module . . . 1
1.2 trainer module . . . 3
1.3 evaluate module . . . 4
1.4 fastmri module . . . 5
1.5 settings module . . . 9
1.6 models package . . . 9
1.7 process package . . . 18

Python Module Index **27**

Index **29**

CHAPTER

ONE

API REFERENCE

1.1 dataset module

dataset.py

This module defines dataset classes for cine MRI segmentation and reconstruction experiments. It provides loaders for raw k-space data, preprocessed k-space arrays, reconstructed images, and joint k-space + image datasets.

Supported modes include: - Directly reading k-space from .mat files with segmentation labels (.nii.gz). - Loading preprocessed k-space slices stored as .npz files. - Loading reconstructed image–mask pairs stored as .npy files. - Providing aligned k-space and image pairs for joint training.

The datasets handle normalization, optional undersampling masks, real/imaginary channel conversion, and alignment of segmentation masks with k-space/image data. These classes are designed to be used with PyTorch DataLoader for training and evaluating deep learning models.

class dataset.**CineSegDataset**(*root*, *cases*, *frames*, *target_shape*, *num_coil=1*, *n_classes=4*, *undersample=False*, *R_list=(24,)*, *acs=2*, *seed=42*)

Bases: Dataset

Dataset for cine MRI segmentation directly from raw .mat k-space files and corresponding NIfTI segmentation labels.

This dataset dynamically loads k-space data (from *.mat*) and segmentation labels (from *.nii.gz*) for each case and frame. It supports optional k-space undersampling with a Cartesian mask and outputs both under-sampled and full-sampled k-space in RI (real/imag) format.

Returns

- **k_us_ri** (*torch.FloatTensor*) – Under-sampled k-space in RI format, shape (2*C, H, W).
- **seg** (*torch.LongTensor*) – Segmentation mask, shape (H, W).
- **k_full_ri** (*torch.FloatTensor*) – Full-sampled k-space in RI format, shape (2*C, H, W).

Parameters

- **args** (*Any*) –
- **kwargs** (*Any*) –

Return type
Any

class dataset.**CineSegPreparedDataset**(*root_dir*, *target_shape*, *undersample=False*, *R_list=(24,)*, *acs=2*, *seed=42*)

Bases: Dataset

Dataset for cine MRI segmentation from preprocessed .npz files.

This dataset loads k-space data and segmentation labels from *.npz* files prepared in advance. Each file stores real and imaginary parts of k-space, along with segmentation masks, aligned per frame. It supports optional k-space undersampling with a Cartesian mask.

Returns

- **k_us_ri** (*torch.FloatTensor*) – Under-sampled k-space in RI format, shape (2*C, H, W).
- **seg_t** (*torch.LongTensor*) – Segmentation mask, shape (H, W).
- **k_full_ri** (*torch.FloatTensor*) – Full-sampled k-space in RI format, shape (2*C, H, W).

Parameters

- **args** (*Any*) –
- **kwargs** (*Any*) –

Return type
Any

Notes

- This dataset assumes *.npz* files are pre-aligned and scaled.
- Unlike *CineSegDataset*, no raw *.mat* or *.nii.gz* files are read at runtime.

class dataset.**KImgSegDataset**(*k_root*, *img_root*, *split='train'*, *target_shape=(192, 448)*, *n_classes=4*, *undersample=False*, *R_list=(24,)*, *acs=2*, *seed=42*)

Bases: Dataset

K-space + Image Segmentation Dataset.

This dataset provides aligned pairs of k-space data, reconstructed images, and segmentation masks for training or evaluating joint reconstruction–segmentation models.

It supports optional undersampling in k-space to simulate accelerated MRI acquisition.

Returns

- **x** (*torch.Tensor*) – Input tensor, concatenating undersampled k-space (real/imag channels) and the image: shape [C_in, H, W].
- **seg_t** (*torch.LongTensor*) – Segmentation mask tensor: shape [H, W].
- **k_full_ri** (*torch.Tensor*) – Fully-sampled k-space (real/imag channels), normalized: shape [2*vc, H, W].

Parameters

- **args** (*Any*) –
- **kwargs** (*Any*) –

Return type
Any

Notes

- Normalization: scaled by its 99.5th percentile (same to previous method) .
- If undersampling is enabled, a random mask is applied to k-space.
- The dataset matches (case, frame) between k-space and image files by parsing filenames, raising an error if no pairs are found.

class dataset.**NpyImageSegDataset**(*root*, *split='train'*, *n_classes=4*)

Bases: Dataset

Dataset for loading image-mask pairs stored in NumPy (.npy) format. Each sample is a pair of an image (float32) and its corresponding segmentation mask (int16). Images are normalized by percentile scaling.

Returns

- **img** (*torch.FloatTensor*) – Normalized image tensor of shape [1, H, W].
- **msk** (*torch.LongTensor*) – Segmentation mask tensor of shape [H, W], with values in [0, n_classes-1].

Parameters

- **args** (*Any*) –
- **kwargs** (*Any*) –

Return type
Any

1.2 trainer module

trainer.py

Training/validation utilities for cine MRI segmentation and reconstruction. This module wraps a PyTorch model, optimizer, and dataloaders to run full epochs with mixed precision, gradient scaling, and simple checkpointing.

1.2.1 Key features

- Supports two batch formats: (k_us, seg) and (k_us, seg, k_full).
- Computes and logs CE loss, Dice loss, optional reconstruction loss, and mIoU.
- Uses AdamW and tqdm progress bars.
- Saves the best model by validation mIoU to the configured output path.

class trainer.**Trainer**(*model*, *train_loader*, *val_loader*, *lr*, *out_dir*, *dev*, *weight*)

Bases: object

Training and validation loop for segmentation/reconstruction models. This class wraps the model, optimizer, and training/validation dataloaders, providing methods for a full training epoch and evaluation.

Notes

- Supports both 2-element batches *(k_us, seg)* and 3-element batches *(k_us, seg, k_full)* depending on whether reconstruction supervision is included.
- Tracks and prints CE, Dice loss, and optional reconstruction loss.
- Saves the best model based on validation mIoU.

Parameters
out_dir (*Path*) –

train_epoch(*epoch*)

validate(*epoch*)

trainer.**metric_mIoU**(*logits*, *mask*)

Return type
float

Parameters

- **logits** (*torch.Tensor*) –
- **mask** (*torch.Tensor*) –

1.3 evaluate module

evaluate.py

This module provides evaluation utilities for cine MRI segmentation models. It computes quantitative metrics on validation or test datasets to assess model performance.

Supported metrics include: - Cross-entropy loss (averaged per image). - Dice coefficient (both soft Dice from loss and hard Dice from predictions). - Mean Intersection-over-Union (mIoU). - Pixel accuracy and confusion matrix. - Average Surface Distance (ASD) per class and overall, based on predicted

and ground-truth segmentation boundaries.

These evaluation tools are designed to align with the training/validation semantics defined in the Trainer and provide detailed performance summaries for each class as well as overall model performance.

evaluate.**evaluate_segmentation**(*model*, *val_loader*, *device*, *num_classes*, *voxel_spacing=(1.0, 1.0)*, *print_per_class=True*)

Return type
Dict[str, float]

Parameters

- **model** (*torch.nn.Module*) –
- **device** (*torch.device*) –
- **num_classes** (*int*) –
- **voxel_spacing** (*Tuple[float, float]*) –
- **print_per_class** (*bool*) –

Metrics computed to MATCH your training/validation code semantics:

- Cross-Entropy: per-image mean, then averaged over images (same as your validate()).
- Mean Dice: computed via your soft Dice (Dice = 1 - dice_loss), then averaged over images.
- mIoU: uses your metric_mIoU(logits, seg) and averages across images (weighted by batch size).
- Pixel Accuracy + Confusion Matrix: from hard predictions (argmax).
- ASD: symmetric average surface distance per class (2D), averaged over images; units from voxel_spacing.

`evaluate.metric_hard_dice(logits, target, exclude_bg=False)`

Compute mean hard Dice score from segmentation logits.

Return type
`float`

Returns
float – Mean Dice score across the batch (averaged over classes and samples).

Parameters

- **logits** (*torch.Tensor*) –
- **target** (*torch.Tensor*) –
- **exclude_bg** (*bool*) –

1.4 fastmri module

fastmri.py

End-to-end k-space reconstruction pretraining on a fastMRI-style dataset. This script loads slice metadata from CSV, reads multicoil k-space from HDF5, applies PCA coil compression and optional k-space augmentations (global phase, linear phase ramp, flips), and trains a lightweight reconstruction network (KspaceRNNEncoder + small CNN head) to predict RSS magnitude images.

Use this as a quick sanity check of model capacity on a different dataset before running the downstream cine segmentation experiments. It handles data prep, training loop (with AMP and grad-clipping), and checkpoint saving in a single, self-contained file.

`class fastmri.CFG(DATA_DIR='G:\\\\FastMRI\\\\Dataset\\\\multicoil_oringin', TRAIN_CSV='G:\\\\FastMRI\\\\Dataset\\\\classify.csv', MAX_VC=4, TARGET_HW=(320, 320), P_PHASE=0.3, NOISE_STD=0.01, BATCH_SIZE=4, EPOCHS=20, LR=0.0002, NUM_WORKER=0, OUT_DIR='runs/recon', SEED=42, FUSE='concat', NF=64, CLIP_NORM=1.0, AMP=True, DEVICE='cuda')`

Bases: `object`

Parameters

- **DATA_DIR** (*str*) –
- **TRAIN_CSV** (*str*) –
- **MAX_VC** (*int*) –
- **TARGET_HW** (*tuple[int, int]*) –
- **P_PHASE** (*float*) –

- **NOISE_STD** (*float*) –
- **BATCH_SIZE** (*int*) –
- **EPOCHS** (*int*) –
- **LR** (*float*) –
- **NUM_WORKER** (*int*) –
- **OUT_DIR** (*str*) –
- **SEED** (*int*) –
- **FUSE** (*str*) –
- **NF** (*int*) –
- **CLIP_NORM** (*float*) –
- **AMP** (*bool*) –
- **DEVICE** (*str*) –

AMP: bool = True

BATCH_SIZE: int = 4

CLIP_NORM: float = 1.0

DATA_DIR: str = 'G:\\FastMRI\\Dataset\\multicoil_oringin'

DEVICE: str = 'cuda'

EPOCHS: int = 20

FUSE: str = 'concat'

LR: float = 0.0002

MAX_VC: int = 4

NF: int = 64

NOISE_STD: float = 0.01

NUM_WORKER: int = 0

OUT_DIR: str = 'runs/recon'

P_PHASE: float = 0.3

SEED: int = 42

TARGET_HW: tuple[int, int] = (320, 320)

TRAIN_CSV: str = 'G:\\FastMRI\\Dataset\\classify.csv'

class fastmri.**EnhancedKspaceSliceReconDataset**(*rows*, *data_dir*, *max_virtual_coils=8*, *target_hw=(320, 320)*, *undersample_R=1*, *acs=0*, *noise_std=0.01*, *train=True*, *p_phase=0.3*, *phase_coilwise=False*, *p_ramp=0.3*, *max_shift_px=8.0*, *p_flipx=0.2*, *p_flipy=0.2*, *seed=42*)

Bases: `Dataset`

Dataset for k-space based image reconstruction pretraining.

- Loads k-space slices from .h5 files.
- Applies PCA coil compression, cropping, and optional data augmentations (global phase shift, linear phase ramp, horizontal/vertical flips).
- For training mode, also applies under-sampling and complex Gaussian noise (augmentation).
- Generates input k-space (real+imag channels) and ground truth image reconstructed from fully-sampled augmented k-space (RSS magnitude).

Returns

- **x** (*torch.FloatTensor*) – Input k-space with real/imag channels, shape [2*vc, H, W].
- **y** (*torch.FloatTensor*) – Target magnitude image, shape [1, H, W].

class `fastmri.`**`KspaceReconNet`**(*in_ch*, *nf=64*, *fuse='concat'*)

Bases: `Module`

Simple reconstruction network for k-space pretraining. - Encodes k-space features with a recurrent encoder (KspaceRNNEncoder). - Decodes to a single-channel magnitude image via small CNN head. - Uses L1 loss between normalized prediction and target image.

Parameters

- **in_ch** (*int*) –
- **nf** (*int*) –
- **fuse** (*str*) –

forward(x):

Forward pass, returns reconstructed image [B,1,H,W].

recon_loss(pred, target_img):

Computes normalized L1 reconstruction loss.

forward(*x*)

Define the computation performed at every call.

Should be overridden by all subclasses.

Note: Although the recipe for forward pass needs to be defined within this function, one should call the `Module` instance afterwards instead of this since the former takes care of running the registered hooks while the latter silently ignores them.

---

static recon_loss(*pred*, *target_img*)

Return type
`Tensor`

Parameters

- **pred** (*Tensor*) –
- **target_img** (*Tensor*) –

training: bool

fastmri.**center_crop_or_pad_k**(*k*, *target_hw*)

Center-crop or zero-pad complex k-space data to a fixed target size.

Return type
ndarray

Parameters

- **k** (*ndarray*) –
- **target_hw** (*Tuple[int, int]*) –

fastmri.**load_train_frames**(*csv_path*)

Load training frame metadata from a CSV file.

Return type
DataFrame

Parameters
csv_path (*str*) –

fastmri.**main**()

Main training loop for k-space reconstruction pretraining.

- Loads training dataset from CSV.
- Builds DataLoader and model.
- Trains for the configured number of epochs.
- Saves the trained model checkpoint and training loss history.

fastmri.**make_cartesian_mask**(*H*, *W*, *R*, *acs=0*, *rng=None*)

Generate a Cartesian undersampling mask along the phase-encoding direction.

Return type
np.ndarray

Parameters

- **H** (*int*) –
- **W** (*int*) –
- **R** (*int*) –
- **acs** (*int*) –
- **rng** (*np.random.RandomState | None*) –

fastmri.**norm01**(*x*, *eps=1e-06*)

Normalize tensor values to the [0,1] range.

Return type
Tensor

Parameters
x (*Tensor*) –

fastmri.**train_one_epoch**(*model*, *loader*, *opt*, *scaler*, *device*)

Run one training epoch for the reconstruction model.

Parameters

- **model** (*nn.Module*) – The reconstruction network.
- **loader** (*DataLoader*) – Training data loader.
- **opt** (*torch.optim.Optimizer*) – Optimizer.
- **scaler** (*torch.amp.GradScaler*) – Mixed precision gradient scaler.
- **device** (*torch.device*) – Training device.

1.5 settings module

1.5.1 settings.py

Centralized configuration file for the KNet project.

This file defines constants and default parameters used across data preparation, training, and evaluation scripts. By storing all settings here, experiments can be reproduced and modified more easily without changing multiple files.

Contents

1. **Data preparation settings** - Paths to raw cine MRI k-space data. - Output directories for reconstructed images (fully sampled and undersampled).
2. **Training settings** - Default training hyperparameters (epochs, batch size, learning rate, etc.). - Dataset split parameters (frames, cases range). - Default checkpoint filenames for different models.
3. **FastMRI settings** - Paths to the FastMRI dataset. - CSV file with labels or metadata.

Usage

Import this file wherever configuration values are required. For example:

```
from settings import DEFAULTS, PATH_K

print(DEFAULTS.epochs)        # 50
print(PATH_K)                 # path to cine MRI ValidationSet

# Override for custom experiments
cfg = DEFAULTS
cfg.epochs = 100
```

1.6 models package

models.modules	module.py
models.k_model	K_model.py
models.k_image_model	K_image_model.py
models.image_model	image_model.py

1.6.1 models.modules

module.py

This file defines core neural network building blocks for k-space and image-based learning tasks. It includes GRU-based sweep encoders, convolutional blocks, and a U-Net implementation that serve as reusable components for segmentation and reconstruction models.

Classes

CoarseHead(*args, **kwargs)	Coarse prediction head for segmentation or reconstruction.
ConvBlock(in_ch, out_ch[, use_in, drop_p])	Convolutional block with optional instance normalization and dropout.
KspaceRNNEncoder(in_ch[, nf, fuse, fuse_drop_p])	Recurrent encoder for k-space features using bidirectional GRU sweeps.
SweepGRU_2(in_ch, hidden, axis[, ...])	Bidirectional GRU sweeper module for spatial feature extraction.
UNet(*args, **kwargs)	U-Net for image segmentation.
UpBlock(in_ch, out_ch[, use_in, drop_p])	U-Net upsampling block with skip connection.

class models.modules.**CoarseHead**(**args*, ***kwargs*)

Bases: Module

Coarse prediction head for segmentation or reconstruction.

This module applies a small convolutional block to reduce feature maps into a single-channel coarse output. It is typically used as an auxiliary head in multi-stage architectures.

Parameters

- **in_ch** (*int*) – Number of input feature channels.
- **Shape** –
- **-------** –
- **Input** (*(B, C, H, W) where C = in_ch*) –
- **Output** (*(B, 1, H, W) coarse prediction map*) –
- **args** (*Any*) –
- **kwargs** (*Any*) –

Return type
Any

forward(*x*)

class models.modules.**ConvBlock**(*in_ch*, *out_ch*, *use_in=True*, *drop_p=0.0*)

Bases: Module

Convolutional block with optional instance normalization and dropout.

This block applies two sequential Conv2d layers with 3x3 kernels, each followed by normalization (instance norm or identity), ReLU activation, and optional spatial dropout.

Parameters

- **in_ch** (*int*) – Number of input channels.
- **out_ch** (*int*) – Number of output channels.
- **use_in** (*bool, default=True*) – If True, applies InstanceNorm2d after each convolution. If False, skips normalization.
- **drop_p** (*float, default=0.0*) – Dropout probability for Dropout2d. If 0, dropout is skipped.
- **Shape** –
- **-----** –
- **Input** (*(B, in_ch, H, W)*) –
- **Output** (*(B, out_ch, H, W)*) –
- **args** (*Any*) –
- **kwargs** (*Any*) –

Return type
Any

forward(*x*)

class models.modules.**KspaceRNNEncoder**(*in_ch*, *nf=64*, *fuse='concat'*, *fuse_drop_p=0.0*)

Bases: Module

Recurrent encoder for k-space features using bidirectional GRU sweeps.

This module encodes k-space features by sweeping bidirectional GRUs along both horizontal and vertical axes, then fuses their outputs. The fusion can be either concatenation followed by a 1x1 convolution, or element-wise addition. In our experiment, we use concatenation.

Parameters

- **in_ch** (*int*) – Number of input channels.
- **nf** (*int, default=64*) – Hidden size of each GRU sweep.
- **fuse** (*{"concat", "add"}, default="concat"*) – Fusion method for horizontal and vertical sweeps: - "concat": concatenate and project with 1x1 convolution. - "add": element-wise addition.
- **fuse_drop_p** (*float, default=0.0*) – Dropout probability applied after fusion.
- **Shape** –
- **-------** –
- **Input** (*(B, C, H, W)*) –
- **Output** (*(B, out_ch, H, W)*) –
- **args** (*Any*) –
- **kwargs** (*Any*) –

Return type
Any

forward(*x*)

class models.modules.**SweepGRU_2**(*in_ch*, *hidden*, *axis*, *num_layers=3*, *residual=True*)

Bases: Module

Bidirectional GRU sweeper module for spatial feature extraction.

This module applies a stack of bidirectional GRU layers along a chosen spatial axis of the input feature map. It is useful for modeling long-range dependencies in k-space.

Parameters

- **in_ch** (*int*) – Number of input feature channels.
- **hidden** (*int*) – Hidden size of the GRU layers.
- **axis** (*int*) – Axis along which to sweep (-1 for width, -2 for height).
- **num_layers** (*int, default=3*) – Number of stacked GRU layers.
- **residual** (*bool, default=True*) – Whether to add residual connections between input and output when dimensions match.
- **Shape** –
- **-----** –
- **Input** (*(B, C, H, W)*) –
- **Output** (*(B, 2*hidden, H, W)*) –
- **args** (*Any*) –
- **kwargs** (*Any*) –

Return type
Any

forward(*x*)

class models.modules.**UNet**(**args*, ***kwargs*)

Bases: Module

U-Net for image segmentation.

This model consists of an encoder-decoder structure with skip connections. The encoder path (downsampling) captures context, while the decoder path (upsampling) restores spatial resolution using skip connections from corresponding encoder layers.

Parameters

- **in_ch** (*int*) – Number of input channels.
- **out_ch** (*int, default=1*) – Number of output channels (e.g., segmentation classes or 1 for binary mask).
- **base** (*int, default=24*) – Number of base feature channels; doubled at each downsampling step.
- **use_in** (*bool, default=True*) – Whether to use InstanceNorm in convolutional blocks.
- **num_pool** (*int, default=4*) – Number of pooling (downsampling) stages in the encoder.
- **Shape** –
- **-----** –
- **Input** (*(B, in_ch, H, W)*) –
- **Output** (*(B, out_ch, H, W)*) –

- **args** (*Any*) –
- **kwargs** (*Any*) –

Return type
Any

forward(*x*)

class models.modules.**UpBlock**(*in_ch*, *out_ch*, *use_in=True*, *drop_p=0.0*)

Bases: Module

U-Net upsampling block with skip connection.

Parameters

- **in_ch** (*int*) – Number of input channels (after concatenation with skip).
- **out_ch** (*int*) – Number of output channels after convolution.
- **use_in** (*bool, default=True*) – Whether to use instance normalization in ConvBlock.
- **drop_p** (*float, default=0.0*) – Dropout probability inside ConvBlock.
- **Shape** –
- **-----** –
- **Input** (*(B, C_in, H_in, W_in) # decoder feature*) –
- **Output** (*(B, out_ch, H_skip, W_skip)*) –
- **args** (*Any*) –
- **kwargs** (*Any*) –

Return type
Any

forward(*x*, *skip*)

1.6.2 models.k_model

K_model.py

This file defines the k-space only segmentation model.

It provides a configuration dataclass and the *KSegModel*, which combines a recurrent k-space encoder, a coarse head, a U-Net refiner, and an auxiliary reconstruction head for self-supervised learning.

For both fully sampling and undersampling

Classes

KSegModel(cfg)	A K-space segmentation model, only use K-space.
ModelCfg([in_ch, nf, num_classes, ...])	

class models.k_model.**KSegModel**(*cfg*)

Bases: Module

A K-space segmentation model, only use K-space.

This model combines a recurrent K-space encoder, a coarse prediction head, a U-Net refiner, and an auxiliary image reconstruction head. It supports segmentation with additional supervision from image reconstruction.

Parameters

- **cfg** (ModelCfg) – Configuration object specifying input channels, number of filters, U-Net base channels, normalization usage, and number of segmentation classes.
- **args** (*Any*) –
- **kwargs** (*Any*) –

Return type
Any

enc

K-space encoder using recurrent layers to extract features.

Type
nn.Module

coarse

Coarse segmentation head operating on encoder features.

Type
nn.Module

refiner

U-Net refiner for final segmentation logits.

Type
nn.Module

recon_head

Auxiliary image reconstruction head for self-supervised supervision.

Type
nn.Sequential

img_rec_logits

Reconstructed image logits (set during forward pass).

Type
torch.Tensor

Loss function

----

loss(logits

Compute combined loss terms: - "ce": Cross-entropy loss for segmentation. - "dice": Dice loss for segmentation. - "recon": L1 reconstruction loss between predicted and ground-truth images. - "total": Weighted sum of segmentation and reconstruction losses.

Type
torch.Tensor, mask: torch.Tensor, kspace: torch.Tensor) -> dict

forward(*k_ri*)

loss(*logits*, *mask*, *kspace*)

class models.k_model.**ModelCfg**(*in_ch=2*, *nf=64*, *num_classes=4*, *unet_base=24*, *use_in=True*)

Bases: object

Parameters

- **in_ch** (*int*) –
- **nf** (*int*) –
- **num_classes** (*int*) –
- **unet_base** (*int*) –
- **use_in** (*bool*) –

in_ch: int = 2

nf: int = 64

num_classes: int = 4

unet_base: int = 24

use_in: bool = True

1.6.3 models.k_image_model

K_image_model.py

This file defines joint k-space and image-domain segmentation models. It includes gating and attention mechanisms to fuse k-space features with image features, enabling multi-modal learning for improved segmentation performance.

For both fully sampling and undersampling

Classes

GateFuse(*args, **kwargs)	Gated fusion module for combining coarse and image features.
KImgSegModel2([k_in_ch, num_classes, nf, ...])	Multi-modal segmentation model combining k-space and image features.

class models.k_image_model.**GateFuse**(**args*, ***kwargs*)

Bases: Module

Gated fusion module for combining coarse and image features.

This module learns a pixel-wise gating map to adaptively fuse two input feature maps.

Parameters

- **args** (*Any*) –
- **kwargs** (*Any*) –

Return type
Any

forward(*coarse*, *img*)

class models.k_image_model.**KImgSegModel2**(*k_in_ch=2*, *num_classes=4*, *nf=64*, *unet_base=24*, *use_in=True*, *drop_img_p=0.0*)

Bases: Module

Multi-modal segmentation model combining k-space and image features.

This model encodes k-space data with a recurrent encoder, generates a coarse reconstruction, and fuses it with image features using channel-wise gating and pixel attention. A U-Net refines the fused representation to produce segmentation outputs.

Parameters

- **k_in_ch** (*int, default=2*) – Number of k-space input channels (e.g., real + imaginary).
- **num_classes** (*int, default=4*) – Number of segmentation classes.
- **nf** (*int, default=64*) – Feature dimension for the k-space encoder.
- **unet_base** (*int, default=24*) – Base number of feature channels for the U-Net.
- **use_in** (*bool, default=True*) – Whether to use Instance Normalization in the U-Net.
- **drop_img_p** (*float, default=0.0*) – Probability of dropping the image channel during training to encourage reliance on k-space features.
- **args** (*Any*) –
- **kwargs** (*Any*) –

Return type
Any

enc

K-space recurrent encoder.

Type
nn.Module

coarse

Coarse reconstruction head producing [B,1,H,W].

Type
nn.Module

se_gate

Squeeze-and-excitation gate for channel-wise modulation.

Type
nn.Module

pix_att

Pixel-level attention mechanism.

Type
nn.Module

refiner

U-Net segmentation head refining fused inputs.

Type
nn.Module

Loss function

----

loss(logits

Compute segmentation losses. Returns a dictionary with: - "ce": Cross-entropy loss. - "dice": Dice loss. - "total": Sum of cross-entropy and Dice losses.

Type
torch.Tensor, target: torch.Tensor, *_) -> dict

forward(*x*)

loss(*logits*, *target*, *_)

1.6.4 models.image_model

image_model.py

This file defines a simple image-domain segmentation model. It wraps the U-Net that is the same as other models for semantic segmentation of reconstructed images, using cross-entropy and Dice loss as the training objectives.

For both fully sampling and undersampling

Classes

ImageSegModel([n_classes, base, use_in])	A lightweight U-Net based segmentation model.

class models.image_model.**ImageSegModel**(*n_classes=4*, *base=24*, *use_in=True*)

Bases: Module

A lightweight U-Net based segmentation model. This model wraps a U-Net backbone for semantic segmentation tasks.

Parameters

- **n_classes** (*int, default=4*) – Number of output classes for segmentation.
- **base** (*int, default=24*) – Base number of feature channels in the U-Net.
- **use_in** (*bool, default=True*) – Whether to use Instance Normalization layers in the U-Net.
- **args** (*Any*) –
- **kwargs** (*Any*) –

Return type
Any

forward(*x*)

loss(*logits*, *target*, *_)

1.7 process package

process.evaluation	evaluation.py
process.k_model_training	k_model_training.py
process.k_image_model_training	k_image_model_training.py
process.image_model_training	image_model_training.py

1.7.1 process.evaluation

evaluation.py

This file provides evaluation routines for segmentation models trained on different input modalities and sampling conditions.

It includes functions to evaluate: - K-space only models (full-sampled and undersampled). - Image-domain models using reconstructed images. - Hybrid models combining k-space and image features.

Functions

K_full_evaluate([cfg])	Evaluate segmentation performance using full-sampled k-space data.
K_image_full_evaluate([cfg])	Evaluate segmentation performance using both k-space and reconstructed full-sampled image features.
K_image_under_evaluate([cfg])	Evaluate segmentation performance using both k-space and reconstructed undersampled image features.
K_under_evaluate([cfg])	Evaluate segmentation performance using undersampled k-space data.
image_full_evaluate([cfg])	Evaluate segmentation performance using reconstructed full-sampled images.
image_under_evaluate([cfg])	Evaluate segmentation performance using reconstructed undersampled images.

Classes

ModelCfg()	

process.evaluation.**K_full_evaluate**(*cfg=namespace(batch=4, cases_from=1, cases_to=60, data_dir=WindowsPath('G:/CMRxRecon2023/pycharm'), data_root=WindowsPath('G:/CMRxRecon2023/ChallengeData_validation/ChallengeDat epochs=50, frames=(0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11), gpu=0, image_full_weight='final_image_full.pt', image_under_weight='final_image_under.pt', k_full_weight='final_k_full.pt', k_image_full_weight='final_k_image_full.pt', k_image_under_weight='final_k_image_under.pt', k_temp='temp.pt', k_under_weight='final_k_under.pt', lr=0.0004, n_classes=4, num_workers=0, out_dir=WindowsPath('runs/checkpoints'), target_h=192, target_w=448)*)

Evaluate segmentation performance using full-sampled k-space data.

Parameters

cfg (*SimpleNamespace, optional*) – Configuration object with attributes such as *gpu*, *batch*, *target_h*, *target_w*, and *num_workers*.

Notes

- Uses CineSegPreparedDataset with fully sampled k-space.
- Loads a model checkpoint trained on full-sampled k-space.
- Reports mIoU, CE loss, Dice loss, and per-class metrics.

`process.evaluation.`**`K_image_full_evaluate`**(*cfg=namespace(batch=4, cases_from=1, cases_to=60, data_dir=WindowsPath('G:/CMRxRecon2023/pycharm'), data_root=WindowsPath('G:/CMRxRecon2023/ChallengeData_validation/Challe epochs=50, frames=(0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11), gpu=0, image_full_weight='final_image_full.pt', image_under_weight='final_image_under.pt', k_full_weight='final_k_full.pt', k_image_full_weight='final_k_image_full.pt', k_image_under_weight='final_k_image_under.pt', k_temp='temp.pt', k_under_weight='final_k_under.pt', lr=0.0004, n_classes=4, num_workers=0, out_dir=WindowsPath('runs/checkpoints'), target_h=192, target_w=448)*)

Evaluate segmentation performance using both k-space and reconstructed full-sampled image features.

Parameters

cfg (*SimpleNamespace, optional*) – Configuration object with *gpu*, *batch*, and dataloader settings.

Notes

- Uses KImgSegDataset combining k-space and reconstructed full-sampled images.
- Loads a hybrid model checkpoint (KImgSegModel2).
- Reports segmentation metrics including per-class scores.

`process.evaluation.`**`K_image_under_evaluate`**(*cfg=namespace(batch=4, cases_from=1, cases_to=60, data_dir=WindowsPath('G:/CMRxRecon2023/pycharm'), data_root=WindowsPath('G:/CMRxRecon2023/ChallengeData_validation/Chall epochs=50, frames=(0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11), gpu=0, image_full_weight='final_image_full.pt', image_under_weight='final_image_under.pt', k_full_weight='final_k_full.pt', k_image_full_weight='final_k_image_full.pt', k_image_under_weight='final_k_image_under.pt', k_temp='temp.pt', k_under_weight='final_k_under.pt', lr=0.0004, n_classes=4, num_workers=0, out_dir=WindowsPath('runs/checkpoints'), target_h=192, target_w=448)*)

Evaluate segmentation performance using both k-space and reconstructed undersampled image features.

Parameters

cfg (*SimpleNamespace, optional*) – Configuration object with GPU index, batch size, and sampling settings.

Notes

- Uses KImgSegDataset combining k-space and reconstructed undersampled images.
- Loads a hybrid model checkpoint trained with undersampling (R=24, ACS=2).
- Reports segmentation metrics including per-class results.

process.evaluation.**K_under_evaluate**(*cfg=namespace(batch=4, cases_from=1, cases_to=60, data_dir=WindowsPath('G:/CMRxRecon2023/pycharm'), data_root=WindowsPath('G:/CMRxRecon2023/ChallengeData_validation/ChallengeDa epochs=50, frames=(0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11), gpu=0, image_full_weight='final_image_full.pt', image_under_weight='final_image_under.pt', k_full_weight='final_k_full.pt', k_image_full_weight='final_k_image_full.pt', k_image_under_weight='final_k_image_under.pt', k_temp='temp.pt', k_under_weight='final_k_under.pt', lr=0.0004, n_classes=4, num_workers=0, out_dir=WindowsPath('runs/checkpoints'), target_h=192, target_w=448)*)

Evaluate segmentation performance using undersampled k-space data.

Parameters

cfg (*SimpleNamespace, optional*) – Configuration object with attributes such as *gpu*, *batch*, *target_h*, *target_w*, *num_workers*, and sampling settings.

Notes

- Uses CineSegPreparedDataset with undersampling (R=24, ACS=2).
- Loads a model checkpoint trained on undersampled k-space.
- Reports segmentation metrics (mIoU, CE loss, Dice loss).

class process.evaluation.**ModelCfg**

Bases: object

in_ch: int = 2

nf: int = 64

num_classes: int = 4

unet_base: int = 24

use_in: bool = True

process.evaluation.**image_full_evaluate**(*cfg=namespace(batch=4, cases_from=1, cases_to=60, data_dir=WindowsPath('G:/CMRxRecon2023/pycharm'), data_root=WindowsPath('G:/CMRxRecon2023/ChallengeData_validation/Challeng epochs=50, frames=(0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11), gpu=0, image_full_weight='final_image_full.pt', image_under_weight='final_image_under.pt', k_full_weight='final_k_full.pt', k_image_full_weight='final_k_image_full.pt', k_image_under_weight='final_k_image_under.pt', k_temp='temp.pt', k_under_weight='final_k_under.pt', lr=0.0004, n_classes=4, num_workers=0, out_dir=WindowsPath('runs/checkpoints'), target_h=192, target_w=448)*)

Evaluate segmentation performance using reconstructed full-sampled images.

Parameters

cfg (*SimpleNamespace, optional*) – Configuration object with GPU index and other evaluation parameters.

Notes

- Uses NpyImageSegDataset with images reconstructed from full k-space.
- Loads a model checkpoint trained on reconstructed full-sampled images.
- Computes and prints segmentation metrics with per-class results.

process.evaluation.**image_under_evaluate**(*cfg=namespace(batch=4, cases_from=1, cases_to=60, data_dir=WindowsPath('G:/CMRxRecon2023/pycharm'), data_root=WindowsPath('G:/CMRxRecon2023/ChallengeData_validation/Challen epochs=50, frames=(0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11), gpu=0, image_full_weight='final_image_full.pt', image_under_weight='final_image_under.pt', k_full_weight='final_k_full.pt', k_image_full_weight='final_k_image_full.pt', k_image_under_weight='final_k_image_under.pt', k_temp='temp.pt', k_under_weight='final_k_under.pt', lr=0.0004, n_classes=4, num_workers=0, out_dir=WindowsPath('runs/checkpoints'), target_h=192, target_w=448)*)

Evaluate segmentation performance using reconstructed undersampled images.

Parameters

cfg (*SimpleNamespace, optional*) – Configuration object with batch size, GPU index, etc.

Notes

- Uses NpyImageSegDataset with images reconstructed from undersampled k-space.
- Loads a model checkpoint trained on reconstructed undersampled images.
- Reports standard segmentation metrics (mIoU, CE loss, Dice loss).

1.7.2 process.k_model_training

k_model_training.py

This script provides training routines for segmentation models that operate purely in the k-space domain. It sets up datasets, builds models, and runs training/validation loops for both fully-sampled and undersampled settings.

Functions

K_full_train([cfg])	Train a segmentation model directly on fully-sampled k-space data.
K_under_train([cfg])	Train a segmentation model directly on undersampled k-space data.

`process.k_model_training.`**`K_full_train`**(*cfg=namespace(batch=4, cases_from=1, cases_to=60, data_dir=WindowsPath('G:/CMRxRecon2023/pycharm'), data_root=WindowsPath('G:/CMRxRecon2023/ChallengeData_validation/Challenge epochs=50, frames=(0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11), gpu=0, image_full_weight='final_image_full.pt', image_under_weight='final_image_under.pt', k_full_weight='final_k_full.pt', k_image_full_weight='final_k_image_full.pt', k_image_under_weight='final_k_image_under.pt', k_temp='temp.pt', k_under_weight='final_k_under.pt', lr=0.0004, n_classes=4, num_workers=0, out_dir=WindowsPath('runs/checkpoints'), target_h=192, target_w=448)*)

Train a segmentation model directly on fully-sampled k-space data. This function prepares the dataset from `prepared/train` and `prepared/val`, builds the k-space based segmentation model, and trains it with the given configuration.

Notes

- Uses k-space as the primary input for segmentation.
- Dataset is based on fully-sampled acquisitions.
- Saves the best model checkpoint according to validation mIoU.

Parameters

cfg (*SimpleNamespace*) –

process.k_model_training.**K_under_train**(*cfg=namespace(batch=4, cases_from=1, cases_to=60, data_dir=WindowsPath('G:/CMRxRecon2023/pycharm'), data_root=WindowsPath('G:/CMRxRecon2023/ChallengeData_validation/Challeng epochs=50, frames=(0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11), gpu=0, image_full_weight='final_image_full.pt', image_under_weight='final_image_under.pt', k_full_weight='final_k_full.pt', k_image_full_weight='final_k_image_full.pt', k_image_under_weight='final_k_image_under.pt', k_temp='temp.pt', k_under_weight='final_k_under.pt', lr=0.0004, n_classes=4, num_workers=0, out_dir=WindowsPath('runs/checkpoints'), target_h=192, target_w=448)*)

Train a segmentation model directly on undersampled k-space data. This function prepares the dataset from `prepared/train` and `prepared/val`, applies undersampling with an acceleration factor, builds the k-space based segmentation model, and trains it with the given configuration.

Notes

- Uses undersampled k-space as the primary input for segmentation.
- Undersampling factor is set via `R_list` and `acs`.
- Saves the best model checkpoint according to validation mIoU.

Parameters
cfg (*SimpleNamespace*) –

1.7.3 process.k_image_model_training

k_image_model_training.py

This script provides training routines for hybrid segmentation models that combine k-space and image-domain features. It supports training with both fully-sampled and undersampled reconstructions, enabling multi-modal learning for improved segmentation.

Functions

K_image_full_train([cfg])	Train a segmentation model using both k-space and fully-sampled image features.
K_image_under_train([cfg])	Train a segmentation model using both k-space and undersampled image features.

process.k_image_model_training.**K_image_full_train**(*cfg=namespace(batch=4, cases_from=1, cases_to=60, data_dir=WindowsPath('G:/CMRxRecon2023/pycharm'), data_root=WindowsPath('G:/CMRxRecon2023/ChallengeData_valida epochs=50, frames=(0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11), gpu=0, image_full_weight='final_image_full.pt', image_under_weight='final_image_under.pt', k_full_weight='final_k_full.pt', k_image_full_weight='final_k_image_full.pt', k_image_under_weight='final_k_image_under.pt', k_temp='temp.pt', k_under_weight='final_k_under.pt', lr=0.0004, n_classes=4, num_workers=0, out_dir=WindowsPath('runs/checkpoints'), target_h=192, target_w=448)*)

Train a segmentation model using both k-space and fully-sampled image features. This function constructs the joint dataset (k-space + image) from `prepared` and `prepared/recon_full`, initializes the hybrid model, and trains it with the given configuration.

Notes

- Uses both k-space and image inputs for training.
- Dataset is based on fully-sampled reconstructions.
- Saves the best model based on validation mIoU.

Parameters

cfg (*SimpleNamespace*) –

process.k_image_model_training.**K_image_under_train**(*cfg=namespace(batch=4, cases_from=1, cases_to=60, data_dir=WindowsPath('G:/CMRxRecon2023/pycharm'), data_root=WindowsPath('G:/CMRxRecon2023/ChallengeData_valid epochs=50, frames=(0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11), gpu=0, image_full_weight='final_image_full.pt', image_under_weight='final_image_under.pt', k_full_weight='final_k_full.pt', k_image_full_weight='final_k_image_full.pt', k_image_under_weight='final_k_image_under.pt', k_temp='temp.pt', k_under_weight='final_k_under.pt', lr=0.0004, n_classes=4, num_workers=0, out_dir=WindowsPath('runs/checkpoints'), target_h=192, target_w=448)*)

Train a segmentation model using both k-space and undersampled image features. This function constructs the joint dataset (k-space + image) from `prepared` and `prepared/recon_undersampled`, applies undersampling, initializes the hybrid model, and trains it with the given configuration.

Notes

- Uses both k-space and image inputs for training.
- Dataset is based on undersampled reconstructions with acceleration factor R.
- Saves the best model based on validation mIoU.

Parameters
cfg (*SimpleNamespace*) –

1.7.4 process.image_model_training

image_model_training.py

This script provides training routines for image-domain segmentation models based on reconstructed MRI images. It loads prepared datasets, constructs U-Net models, and trains them on both fully-sampled and undersampled reconstructions.

Functions

image_full_train([cfg])	Train an image-only segmentation model using fully-sampled reconstructions.
image_under_train([cfg])	Train an image-only segmentation model using undersampled reconstructions.

`process.image_model_training.`**`image_full_train`**(*cfg=namespace(batch=4, cases_from=1, cases_to=60, data_dir=WindowsPath('G:/CMRxRecon2023/pycharm'), data_root=WindowsPath('G:/CMRxRecon2023/ChallengeData_validation/ epochs=50, frames=(0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11), gpu=0, image_full_weight='final_image_full.pt', image_under_weight='final_image_under.pt', k_full_weight='final_k_full.pt', k_image_full_weight='final_k_image_full.pt', k_image_under_weight='final_k_image_under.pt', k_temp='temp.pt', k_under_weight='final_k_under.pt', lr=0.0004, n_classes=4, num_workers=0, out_dir=WindowsPath('runs/checkpoints'), target_h=192, target_w=448)*)

Train an image-only segmentation model using fully-sampled reconstructions. This function loads the prepared dataset from `prepared/recon_full`, builds the UNet-based segmentation model, and trains it with the specified configuration.

Notes

- Uses cross-entropy and dice loss for optimization.
- Saves the best model based on validation mIoU.
- Batch size is fixed at 4 for full-sampled training.

Parameters
cfg (*SimpleNamespace*) –

`process.image_model_training.`**`image_under_train`**(*cfg=namespace(batch=4, cases_from=1, cases_to=60, data_dir=WindowsPath('G:/CMRxRecon2023/pycharm'), data_root=WindowsPath('G:/CMRxRecon2023/ChallengeData_validation epochs=50, frames=(0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11), gpu=0, image_full_weight='final_image_full.pt', image_under_weight='final_image_under.pt', k_full_weight='final_k_full.pt', k_image_full_weight='final_k_image_full.pt', k_image_under_weight='final_k_image_under.pt', k_temp='temp.pt', k_under_weight='final_k_under.pt', lr=0.0004, n_classes=4, num_workers=0, out_dir=WindowsPath('runs/checkpoints'), target_h=192, target_w=448)*)

Train an image-only segmentation model using undersampled reconstructions. This function loads the prepared dataset from `prepared/recon_undersampled`, builds the UNet-based segmentation model, and trains it with the specified configuration.

Notes

- Uses cross-entropy and dice loss for optimization.
- Saves the best model based on validation mIoU.
- Batch size is configurable via `cfg.batch`.

Parameters
cfg (*SimpleNamespace*) –

PYTHON MODULE INDEX

e

evaluate, 4

f

fastmri, 5

m

models, 9
models.image_model, 17
models.k_image_model, 15
models.k_model, 13
models.modules, 10

p

process, 18
process.evaluation, 18
process.image_model_training, 25
process.k_image_model_training, 23
process.k_model_training, 22

s

settings, 9

t

trainer, 3

INDEX

A

AMP (*fastmri.CFG attribute*), 6

B

BATCH_SIZE (*fastmri.CFG attribute*), 6

C

center_crop_or_pad_k() (*in module fastmri*), 8
CFG (*class in fastmri*), 5
CineSegDataset (*class in dataset*), 1
CineSegPreparedDataset (*class in dataset*), 1
CLIP_NORM (*fastmri.CFG attribute*), 6
coarse (*models.k_image_model.KImgSegModel2 attribute*), 16
coarse (*models.k_model.KSegModel attribute*), 14
CoarseHead (*class in models.modules*), 10
ConvBlock (*class in models.modules*), 10

D

DATA_DIR (*fastmri.CFG attribute*), 6
dataset
    module, 1
DEVICE (*fastmri.CFG attribute*), 6

E

enc (*models.k_image_model.KImgSegModel2 attribute*), 16
enc (*models.k_model.KSegModel attribute*), 14
EnhancedKspaceSliceReconDataset (*class in fastmri*), 6
EPOCHS (*fastmri.CFG attribute*), 6
evaluate
    module, 4
evaluate_segmentation() (*in module evaluate*), 4

F

fastmri
    module, 5
forward() (*fastmri.KspaceReconNet method*), 7
forward() (*models.image_model.ImageSegModel method*), 17
forward() (*models.k_image_model.GateFuse method*), 16
forward() (*models.k_image_model.KImgSegModel2 method*), 17
forward() (*models.k_model.KSegModel method*), 14
forward() (*models.modules.CoarseHead method*), 10
forward() (*models.modules.ConvBlock method*), 11
forward() (*models.modules.KspaceRNNEncoder method*), 11
forward() (*models.modules.SweepGRU_2 method*), 12
forward() (*models.modules.UNet method*), 13
forward() (*models.modules.UpBlock method*), 13
FUSE (*fastmri.CFG attribute*), 6

G

GateFuse (*class in models.k_image_model*), 15

I

image_full_evaluate() (*in module process.evaluation*), 20
image_full_train() (*in module process.image_model_training*), 25
image_under_evaluate() (*in module process.evaluation*), 21
image_under_train() (*in module process.image_model_training*), 26
ImageSegModel (*class in models.image_model*), 17
img_rec_logits (*models.k_model.KSegModel attribute*), 14
in_ch (*models.k_model.ModelCfg attribute*), 15
in_ch (*process.evaluation.ModelCfg attribute*), 20

K

K_full_evaluate() (*in module process.evaluation*), 18
K_full_train() (*in module process.k_model_training*), 22
K_image_full_evaluate() (*in module process.evaluation*), 19
K_image_full_train() (*in module process.k_image_model_training*), 23
K_image_under_evaluate() (*in module process.evaluation*), 19

K_image_under_train() (*in module process.k_image_model_training*), 24
K_under_evaluate() (*in module process.evaluation*), 20
K_under_train() (*in module process.k_model_training*), 22
KImgSegDataset (*class in dataset*), 2
KImgSegModel2 (*class in models.k_image_model*), 16
KSegModel (*class in models.k_model*), 13
KspaceReconNet (*class in fastmri*), 7
KspaceRNNEncoder (*class in models.modules*), 11

L

load_train_frames() (*in module fastmri*), 8
loss() (*models.image_model.ImageSegModel method*), 17
loss() (*models.k_image_model.KImgSegModel2 method*), 17
loss() (*models.k_model.KSegModel method*), 15
LR (*fastmri.CFG attribute*), 6

M

main() (*in module fastmri*), 8
make_cartesian_mask() (*in module fastmri*), 8
MAX_VC (*fastmri.CFG attribute*), 6
metric_hard_dice() (*in module evaluate*), 5
metric_mIoU() (*in module trainer*), 4
ModelCfg (*class in models.k_model*), 15
ModelCfg (*class in process.evaluation*), 20
models
    module, 9
models.image_model
    module, 17
models.k_image_model
    module, 15
models.k_model
    module, 13
models.modules
    module, 10
module
    dataset, 1
    evaluate, 4
    fastmri, 5
    models, 9
    models.image_model, 17
    models.k_image_model, 15
    models.k_model, 13
    models.modules, 10
    process, 18
    process.evaluation, 18
    process.image_model_training, 25
    process.k_image_model_training, 23
    process.k_model_training, 22
    settings, 9
    trainer, 3

N

NF (*fastmri.CFG attribute*), 6
nf (*models.k_model.ModelCfg attribute*), 15
nf (*process.evaluation.ModelCfg attribute*), 20
NOISE_STD (*fastmri.CFG attribute*), 6
norm01() (*in module fastmri*), 8
NpyImageSegDataset (*class in dataset*), 3
num_classes (*models.k_model.ModelCfg attribute*), 15
num_classes (*process.evaluation.ModelCfg attribute*), 20
NUM_WORKER (*fastmri.CFG attribute*), 6

O

OUT_DIR (*fastmri.CFG attribute*), 6

P

P_PHASE (*fastmri.CFG attribute*), 6
pix_att (*models.k_image_model.KImgSegModel2 attribute*), 16
process
    module, 18
process.evaluation
    module, 18
process.image_model_training
    module, 25
process.k_image_model_training
    module, 23
process.k_model_training
    module, 22

R

recon_head (*models.k_model.KSegModel attribute*), 14
recon_loss() (*fastmri.KspaceReconNet static method*), 7
refiner (*models.k_image_model.KImgSegModel2 attribute*), 16
refiner (*models.k_model.KSegModel attribute*), 14

S

se_gate (*models.k_image_model.KImgSegModel2 attribute*), 16
SEED (*fastmri.CFG attribute*), 6
settings
    module, 9
SweepGRU_2 (*class in models.modules*), 11

T

TARGET_HW (*fastmri.CFG attribute*), 6
TRAIN_CSV (*fastmri.CFG attribute*), 6
train_epoch() (*trainer.Trainer method*), 4
train_one_epoch() (*in module fastmri*), 8

trainer
    module, 3
Trainer (*class in trainer*), 3
training (*fastmri.KspaceReconNet attribute*), 7

U

UNet (*class in models.modules*), 12
unet_base (*models.k_model.ModelCfg attribute*), 15
unet_base (*process.evaluation.ModelCfg attribute*), 20
UpBlock (*class in models.modules*), 13
use_in (*models.k_model.ModelCfg attribute*), 15
use_in (*process.evaluation.ModelCfg attribute*), 20

V

validate() (*trainer.Trainer method*), 4